

Reinforcement learning for Event Selection in particle physics (LHC)

Rithika Shyam Kumar, University of Bern

1. Problem Statement

At CERN's Large Hadron Collider (LHC), proton-proton collisions occur at 40 million events per second, generating roughly 1 MB of raw detector data each. Only approximately 1,000 events per second can be permanently stored, meaning the trigger system must reject 99.997% of all collisions in real time.

The fundamental challenge is extreme class imbalance: rare physics processes such as Higgs boson production occur at roughly one event per second, against a background of 10^9 inelastic collisions per second. Missing a signal event is irreversible — unlike background events, signal data cannot be recovered.

Current trigger systems rely on fixed, hand-tuned threshold rules. These suffer from three key limitations:

- **Static rules:** They cannot adapt to changing experimental conditions such as increasing pileup (multiple simultaneous collisions).
- **No learning:** Thresholds are set before data-taking begins, based on simulation rather than real observations.
- **No budget awareness:** They apply identical selection criteria regardless of how much storage capacity has been consumed.

This project investigates whether a reinforcement learning agent can overcome these limitations by learning an adaptive, budget-aware selection policy from experience.

2. Problem Formalisation as an MDP

We model the event selection task as a Markov Decision Process (MDP) with the following components.

2.1 State Space

The state at each timestep is a two-element discrete observation:

```
s_t = (signal_score_bin, budget_remaining_bin)
```

```
signal_score_bin ∈ {0, ..., 9} – discretised composite event quality  
budget_remaining_bin ∈ {0, ..., 9} – fraction of storage still available
```

Including the budget fraction in the state is the key structural choice that allows budget-aware decisions. A static threshold has no equivalent: it is stateless.

2.2 Action Space

At each step the agent selects one of two actions:

```
A = {0 = REJECT, 1 = ACCEPT}
```

2.3 Reward Function

The reward function encodes all three objectives — signal efficiency, background suppression, and budget enforcement — in a single scalar:

```
r (s, a) =  
+R_sig      if a = ACCEPT and event is signal (+10.0)  
-R_miss     if a = REJECT and event is signal (-15.0)  
-R_bkg      if a = ACCEPT and event is background (-1.0)  
-R_overbudget if a = ACCEPT and budget exhausted (-5.0)  
0          if a = REJECT and event is background
```

The asymmetry $R_{\text{miss}} > R_{\text{sig}}$ ($15 > 10$) reflects the physical reality that missed signal events are irretrievable. The over-budget penalty enforces the hard storage constraint. Together these terms shape the agent toward the operational objective: keep all signal, reject most background, respect the storage budget.

2.4 Transition Dynamics

Each step advances to the next collision event (independently sampled). The accepted event count increments by 1 if $a = \text{ACCEPT}$ and budget remains. The episode terminates after 1,000 events or when the storage budget (100 accepts) is exhausted.

3. Algorithm: Tabular Q-Learning

I used tabular Q-learning, a model-free, off-policy RL algorithm. It is appropriate here because the discretised state space ($10 \times 10 = 100$ states) is small enough for a full lookup table.

However, this does not mimic the real event and this project can be further extended to using a **Deep Q-Networks (DQN)** or a policy gradient method like **PPO**, since the raw detector observables (transverse momentum, missing energy, jet multiplicity, etc.) form a continuous,

high-dimensional state space that a lookup table cannot represent. These methods use neural networks to approximate the Q-function or policy directly over raw features, eliminating the lossy discretisation step and allowing the agent to generalise across the full continuous state space.

3.1 Q-Table

The Q-function is stored as a 3-dimensional array:

```
Q : {0,...,9} × {0,...,9} × {0,1} → ℝ  
  
Q[score_bin, budget_bin, action] — initialised to zero
```

3.2 Update Rule

After each transition (s, a, r, s'), the Q-table is updated using the Bellman equation:

```
Q(s,a) ← Q(s,a) + α · [ r + γ · max_{a'} Q(s', a') - Q(s, a) ]  
  
α = 0.1      (learning rate)  
γ = 0.99     (discount factor)
```

The TD error in brackets measures the gap between the current estimate and the Bellman target. The update moves the estimate toward the target at rate α .

3.3 Exploration: ϵ -Greedy

```
With probability ε: select random action  (explore)  
With probability 1-ε: select argmax_a Q(s, a)  (exploit)  
  
ε decays from 1.0 → 0.05 multiplicatively (factor 0.995 per episode)
```

This schedule ensures full exploration early in training and near-greedy exploitation by convergence.

4. Results

I trained the agent for 2,000 episodes (1,000 events each, storage budget 100 per episode). After training, the greedy policy was evaluated over 200 fresh episodes and compared to a fixed-threshold baseline ($\tau = 0.55$, chosen as reasonable midpoint between the signal and background score distributions) that accepts any event with score ≥ 0.55 .

Metric	RL Agent	Fixed Threshold	Advantage
--------	----------	-----------------	-----------

Signal Efficiency	~0.85	~0.60	RL +42%
Background Kept	~85	~100	RL -15%
Budget Adaptivity	Yes	No	RL only

4.1 Training Convergence

Episode reward increases steadily through the first ~500 episodes as ϵ decays and the agent learns to prefer high-score events. Signal efficiency rises from near-random (~50%) to consistently above 80%.

4.2 Comparison with Baseline

The fixed-threshold baseline achieves lower signal efficiency because it does not adapt to budget consumption — it often exhausts storage early on lower-quality events, leaving no capacity when high-quality events arrive later in the episode. The RL agent learns to reserve budget for events most likely to be signal.

5. Conclusion and Future Work

I demonstrated that modelling real-time trigger event selection as an MDP and solving it with tabular Q-learning produces a policy that outperforms fixed-threshold systems on signal efficiency while respecting storage constraints. The MDP framing is the key contribution: by encoding budget-fraction in the state and penalising over-budget accepts, the agent learns qualitatively different behaviour — dynamic threshold adaptation — that is structurally impossible for any stateless rule.

Future directions include: (1) replacing the tabular Q-function with linear function approximation over raw physics features to remove the need for manual discretisation; (2) testing on simulated ATLAS/CMS detector data; (3) incorporating episode-level resource constraints through constrained RL formulations.

6. References

1. Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
2. Towards a Self-Driving Trigger at the LHC: Adaptive Response in Real Time, <https://arxiv.org/abs/2601.08910>
3. Duarte, J. et al. (2018). Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation*, 13, P07027. ([CERN server](#))
4. LLM usage: Claude was used for coding assistance and understanding certain concepts